Small Radio Telescope Drive Controller

Technical Documentation and Operations Manual

Version 1.1

Acre Road Observatory
University of Glasgow

March 2026

Abstract

This document provides comprehensive technical documentation for the Small Radio Telescope (SRT) Drive Controller system designed for 21 cm hydrogen line observations at Acre Road Observatory, Glasgow. The system comprises a multi-layered architecture incorporating an Arduino Due microcontroller for low-level motor control, an ESP32-S3 for network interface and coordinate transformations, and a GNU Radio-based receiver with web scheduling capabilities. This document details the hardware configuration, software architecture, communication protocols, coordinate transformation algorithms, installation procedures, and operational guidelines necessary for the deployment and maintenance of the telescope control system.

Contents

1 Introduction

1.1 Purpose and scope

The SRT Drive Controller is a complete control system for an alt-azimuth mounted radio telescope operating at 1420.405 MHz (the hydrogen line). The system enables both manual and automated observations, supporting:

- Real-time telescope positioning via web interface
- Integration with planetarium software (Stellarium)
- Automated coordinate conversion from celestial to horizontal coordinates
- Scheduled observations with data acquisition
- Sun and Moon tracking capabilities

1.2 System overview

The telescope control system employs a layered architecture separating concerns across multiple processing units:

1. **Arduino Due** — Low-level motor control, position encoding, limit switch monitoring, and current sensing
2. **ESP32-S3** — Network interface, astronomical coordinate transformations, web server, and Stellarium protocol implementation
3. **H1 Receiver** — GNU Radio-based spectrum analyser for 21 cm data acquisition
4. **Observation Scheduler** — Web-based automation coordinating telescope pointing and data recording

1.3 Document organisation

This document is organised as follows: Section 2 presents the system architecture; Section 3 details the hardware components; Section 4 describes the Arduino Due firmware; Section 5 covers the ESP32 controller; Section 6 explains the coordinate transformation algorithms; Section 7 describes the receiver and scheduler; Section 8 provides installation procedures; Section 9 covers operational procedures; and Section 10 addresses common issues.

2 System architecture

2.1 Architectural overview

The system architecture follows a hierarchical design principle, with each layer responsible for specific functionality. Figure 1 illustrates the component relationships and communication paths.

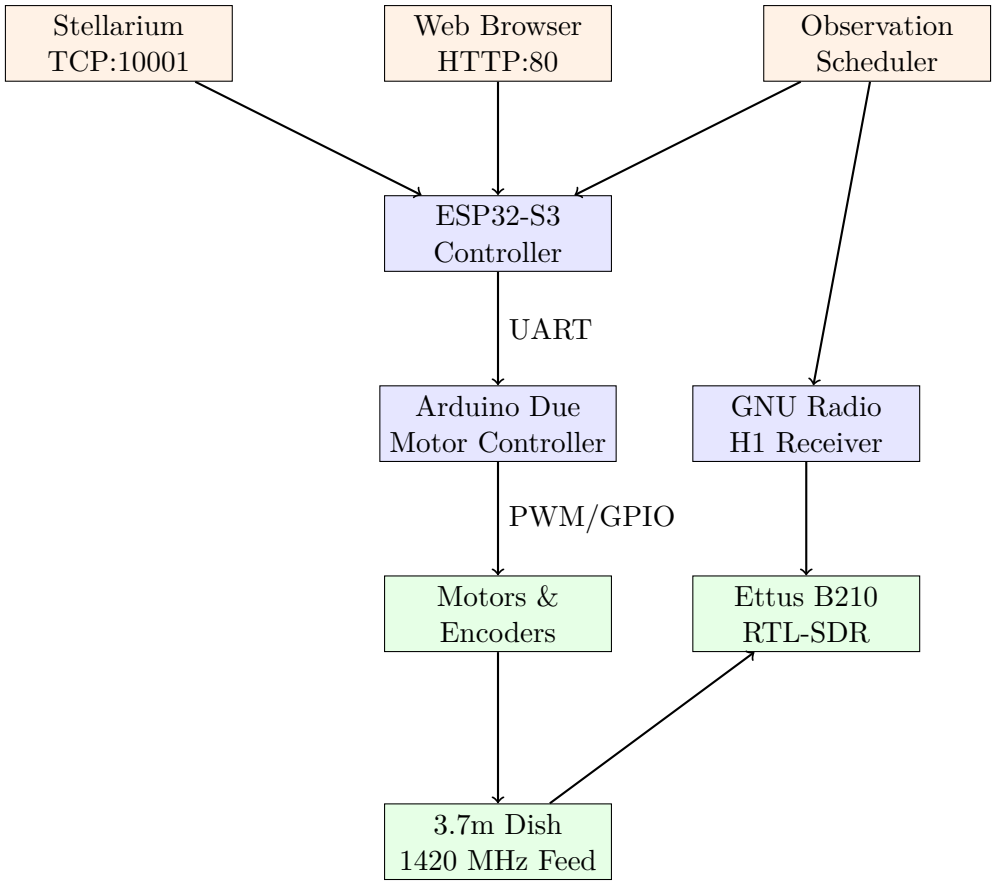

Figure 1: System architecture showing component relationships and communication interfaces

2.2 Data flow

The primary data flow for a typical observation proceeds as follows:

1. User or scheduler specifies target coordinates (RA/Dec, Galactic, or direct Alt/Az)
2. ESP32 converts celestial coordinates to horizontal (Alt/Az) using current sidereal time
3. ESP32 transmits target position to Arduino Due via UART
4. Arduino Due calculates motion profile and drives motors with PWM
5. Reed switch encoders provide position feedback
6. Due reports position and status to ESP32
7. ESP32 updates web interface and Stellarium position
8. Receiver captures spectrum data once telescope is on target

2.3 Communication interfaces

Table 1 summarises the communication interfaces employed throughout the system.

Table 1: System communication interfaces

Interface	Protocol	Speed	Purpose
Due ↔ ESP32	UART Serial	115200 baud	Position commands, status
ESP32 ↔ Browser	HTTP	TCP:80	Web interface, API
ESP32 ↔ Stellarium	TCP	Port 10001	Telescope protocol
Due ↔ USB	Serial	115200 baud	Debug, direct control
ESP32 WiFi	IEEE 802.11	—	Network connectivity
ESP32 Ethernet	10/100 Mbps	W5500 SPI	Wired network

3 Hardware configuration

3.1 Arduino Due microcontroller

The Arduino Due serves as the real-time motor control platform, selected for its:

- ARM Cortex-M3 processor (84 MHz) with deterministic interrupt handling
- 12-bit analogue-to-digital converters for current sensing
- Multiple hardware UART interfaces
- PWM outputs with adequate resolution for smooth motor control
- 3.3 V logic levels compatible with the ESP32

3.1.1 Pin assignments

Table 2 details the complete pin assignment for the Arduino Due.

Table 2: Arduino Due pin assignments

Function	Pin	Wire Colour	Notes
Fault Flags (inputs)			
Az Fault Flag 1	5	Blue	Motor driver status
Az Fault Flag 2	4	Purple	Motor driver status
Alt Fault Flag 1	7	Blue	Motor driver status
Alt Fault Flag 2	6	Purple	Motor driver status
Motor Control (outputs)			
Az PWM	8	Green	Speed control (inverted)
Az Direction	9	Yellow	LOW=West, HIGH=East
Alt PWM	10	Green	Speed control (inverted)
Alt Direction	11	Yellow	LOW=Down, HIGH=Up
Driver Reset (outputs)			
Az Reset	22	Black	HIGH=enabled
Alt Reset	24	White	HIGH=enabled
Position Encoders (inputs)			
Az Pulses	12	Blue	Reed switch, falling edge
Alt Pulses	13	Orange	Reed switch, falling edge
Current Sensors (analogue inputs)			
Az Current	A1	Black	ACS712 hall-effect
Alt Current	A0	White	ACS712 hall-effect
Serial1 (ESP32 interface)			
TX1	18	—	To ESP32 RX
RX1	19	—	From ESP32 TX

3.2 Motor drive system

3.2.1 H-bridge motor drivers

The system employs H-bridge motor drivers with the following characteristics:

- Dual fault flag outputs (FF1, FF2) indicating:
  - FF1=LOW, FF2=HIGH: Short circuit
  - FF1=HIGH, FF2=LOW: Overheating
  - FF1=HIGH, FF2=HIGH: Undervoltage
- Active-low reset input
- PWM speed control with inverted logic (255 = stopped, 0 = full speed)
- Direction control input

3.2.2 Position encoders

Position feedback is provided by reed switch encoders mounted on each axis:

- Resolution: 2 pulses per degree (0.5° positioning accuracy)
- Detection: Falling edge triggered interrupts
- Debouncing: 15 ms minimum interval between valid pulses

3.2.3 Current sensing

Motor current is monitored using ACS712 hall-effect current sensors:

- Zero-current output voltage: 2.5 V
- Sensitivity: 66 mV/A
- Default overcurrent threshold: 5.0 A

The current reading is computed as:

$$I = \frac{V_{\text{ADC}} - V_{\text{offset}}}{S} \tag{1}$$

where V_{ADC} is the measured voltage, $V_{\text{offset}} = 2.5\,\text{V}$, and $S = 0.066\,\text{V/A}$.

3.3 ESP32-S3 controller

The ESP32-S3 provides the network interface and computational resources for coordinate transformations:

- Dual-core Xtensa LX7 processor
- WiFi 802.11 b/g/n (2.4 GHz)
- 4 MB flash, 512 KB SRAM
- MicroPython runtime environment

3.3.1 ESP32 to Arduino Due wiring

Table 3: ESP32-S3 to Arduino Due serial connection

ESP32-S3	**Arduino Due**	**Function**
GPIO17	Pin 19 (RX1)	ESP32 TX → Due RX
GPIO18	Pin 18 (TX1)	ESP32 RX ← Due TX
GND	GND	Common ground

3.3.2 W5500 Ethernet module (optional)

For installations requiring wired network connectivity:

Table 4: W5500 Ethernet module wiring

W5500	**ESP32-S3**	**Function**
MOSI	GPIO11	SPI data out
MISO	GPIO13	SPI data in
SCK	GPIO12	SPI clock
CS	GPIO10	Chip select
RST	GPIO9	Reset
3.3V	3.3V	Power
GND	GND	Ground

3.4 Operational limits

Table 5: Telescope position limits

Axis	Minimum	Maximum
Altitude (hardware)	0° (horizon)	90° (zenith)
Altitude (software)	0°	90°
Azimuth (hardware)	0° (North)	355°
Azimuth (software)	2°	353°

The two-tier limit system provides a 2° safety margin between software limits and physical stops.

4 Arduino Due firmware

4.1 Firmware overview

The Arduino Due firmware (`src/main.cpp`) implements a state machine-based motor controller with the following responsibilities:

- Automatic homing sequence on startup
- Smooth motion profiles with acceleration/deceleration ramps
- Position tracking via interrupt-driven encoder counting
- Safety monitoring (overcurrent, stall detection, limit switches)
- Serial command interface
- Configuration persistence to flash memory

4.2 State machine

The controller operates as a finite state machine with five primary states:

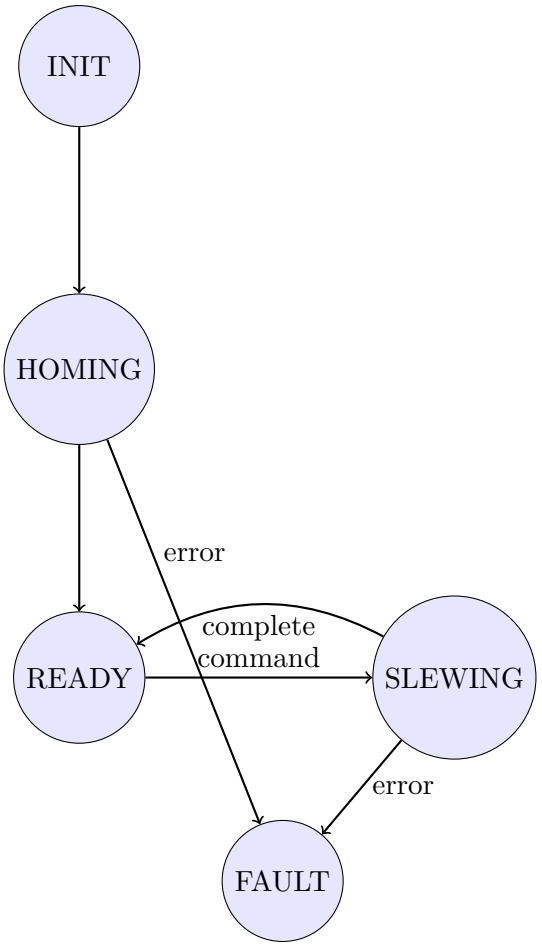

Figure 2: Controller state machine diagram

INIT Hardware initialisation, GPIO configuration, driver enabling

HOMING Driving to limit switches, establishing position reference

READY Idle state, accepting commands

SLEWING Active motion toward target position

FAULT Error condition, motors disabled

4.3 Homing sequence

The homing procedure establishes the absolute position reference:

1. Both motors drive toward their respective limit switches (azimuth west, altitude down)
2. When no encoder pulses are received for 2 seconds (stall timeout), the motor is considered at the limit
3. Position counters are reset to zero
4. Motors drive to the home position (default: Alt=0°, Az=180°)
5. System transitions to READY state

4.4 Motion profiles

Smooth motion profiles prevent structural oscillation and mechanical stress:

4.4.1 Acceleration ramp

The PWM value ramps linearly from minimum speed to full speed over the configured ramp-up time (default 500 ms):

$$\text{PWM}(t) = \text{PWM}_{\text{min}} + \frac{(\text{PWM}_{\text{full}} - \text{PWM}_{\text{min}}) \cdot t}{T_{\text{ramp}}} \tag{2}$$

where $\text{PWM}_{\text{min}} = 220$ (minimum speed), $\text{PWM}_{\text{full}} = 0$ (full speed), and T_{ramp} is the ramp-up time.

4.4.2 Deceleration ramp

Deceleration uses a quadratic profile based on remaining distance to target (default 7°):

$$\text{PWM}(d) = \text{PWM}_{\text{full}} + (\text{PWM}_{\text{min}} - \text{PWM}_{\text{full}}) \cdot \left(1 - \frac{d}{d_{\text{ramp}}}\right)^2 \tag{3}$$

where d is the remaining distance and d_{ramp} is the deceleration distance.

4.5 Serial command interface

The firmware accepts commands via USB serial (Programming Port) and Serial1 (ESP32 connection) at 115200 baud.

4.5.1 Motion commands

Table 6: Motion commands

Command	Description
`<alt> <az>`	Slew to position (e.g., `45.0 180.0`)
`DRIVE <alt> <az>`	Slew to position (explicit form)
`HOME`	Run homing sequence
`STOP`	Emergency stop
`RESET`	Clear fault and re-home

4.5.2 Configuration commands

Table 7: Configuration commands

Command	Description
`STATUS`	Show current position and status
`CONFIG`	Show all configuration parameters
`SET <param> <value>`	Set configuration parameter
`SAVE`	Save configuration to flash
`LOAD`	Load configuration from flash
`DEFAULTS`	Reset to factory defaults
`HELP`	Show command help

4.5.3 Configurable parameters

Table 8: SET command parameters

Parameter	Description	Default	Unit
ALTMIN	Altitude software minimum	0	degrees
ALTMAX	Altitude software maximum	90	degrees
AZMIN	Azimuth software minimum	2	degrees
AZMAX	Azimuth software maximum	353	degrees
HOMEALT	Home altitude position	0	degrees
HOMEAZ	Home azimuth position	180	degrees
RAMPUP	Acceleration time	500	ms
RAMPDOWN	Deceleration distance	7	degrees
STOPRAMP	Reversal deceleration time	300	ms
CURRENT	Overcurrent threshold	5.0	Amps
STALL	Stall detection timeout	2000	ms

4.6 Status output format

The controller outputs status in a machine-parseable format:

```
Alt:45.0 Az:180.0 Ialt:0.52A Iaz:0.38A Status:Ready
Alt:30.0 Az:150.0 Ialt:2.10A Iaz:1.85A Status:Slewing -> Alt:45.0 Az:180.0
Alt:30.0 Az:150.0 Ialt:5.20A Iaz:0.00A Status:FAULT [Altitude motor overcurrent]
```

Listing 1: Status output examples

4.7 Fault detection

The system monitors multiple fault conditions:

Table 9: Fault conditions and detection

Fault	Detection	Likely Cause
Short circuit	FF1=LOW, FF2=HIGH	Wiring fault, motor failure
Overheating	FF1=HIGH, FF2=LOW	Prolonged high current
Undervoltage	FF1=HIGH, FF2=HIGH	Power supply issue
Overcurrent	Current > 5.0 A	Obstruction, binding
Stall	No pulses for 2 s	Motor failure, jam
Position bounds	Exceeds limits	Encoder error

4.8 Simulation mode

For testing without hardware, the firmware can be built with the `SIMULATION_MODE` flag:

```
pio run -e simulation
```

In simulation mode:

- All GPIO operations are replaced with software stubs
- A `simulatePulses()` function generates position feedback based on PWM commands
- Current sensors report 0 A

- The complete control loop operates as it would with real hardware

Simulation parameters (configurable in `config.h`):

Table 10: Simulation mode parameters

Parameter	Default	Description
`SIM_MAX_SPEED_DEG_S`	18.0	Maximum speed (deg/s)
`SIM_INITIAL_AZ_DEG`	180.0	Starting azimuth
`SIM_INITIAL_ALT_DEG`	45.0	Starting altitude

5 ESP32 controller

5.1 Software architecture

The ESP32 controller runs MicroPython with a modular architecture:

Table 11: ESP32 MicroPython modules

Module	Purpose	Size
`boot.py`	WiFi/Ethernet initialisation	∼0.5 KB
`main.py`	Main application, tracking loop	∼4.5 KB
`config.py`	Configuration settings	∼1.2 KB
`coordinates.py`	Astronomical transforms	∼17 KB
`web_server.py`	HTTP server and web UI	∼34 KB
`stellarium.py`	Stellarium telescope protocol	∼3.5 KB
`srt_serial.py`	Serial communication with Due	∼5 KB
`wifi_manager.py`	WiFi AP/station management	∼4 KB
`ethernet.py`	W5500 Ethernet management	∼5 KB

5.2 Configuration

Key configuration parameters in `config.py`:

```
# WiFi Access Point
WIFI_AP_SSID = "SRT_Controller"
WIFI_AP_PASSWORD = "radio1420"

# Serial connection to Arduino Due
DUE_UART_TX = 17
DUE_UART_RX = 18
DUE_BAUD_RATE = 115200

# Observer location (critical for coordinate transforms)
OBSERVER_LAT = 55.9    # Latitude (degrees, north positive)
OBSERVER_LON = -4.3    # Longitude (degrees, east positive)

# Server ports
STELLARIUM_PORT = 10001
WEB_PORT = 80

# Mount software limits (must match Due configuration)
MOUNT_AZ_MIN = 2.0
MOUNT_AZ_MAX = 353.0
MOUNT_ALT_MIN = 0.0
```

```
MOUNT_ALT_MAX = 90.0
```

Listing 2: ESP32 configuration parameters

5.3 Networking

5.3.1 Network priority

The ESP32 supports multiple network interfaces with the following priority:

1. **Ethernet** (if enabled and connected) — Preferred for Stellarium
2. **WiFi Station** (if connected to saved network)
3. **WiFi Access Point** (always active at 192.168.4.1)

5.3.2 Default network settings

Table 12: Default network configuration

Setting	Value
WiFi AP SSID	SRT_Controller
WiFi AP Password	radio1420
AP IP Address	192.168.4.1
Web Server Port	80
Stellarium Port	10001

5.4 HTTP API

The ESP32 provides a RESTful HTTP API for telescope control.

5.4.1 Status endpoints

Table 13: HTTP status endpoints

Endpoint	Method	Description
`/`	GET	Web interface (HTML)
`/status`	GET	Mount position and status (JSON)
`/tracking`	GET	Current tracking state
`/ephemeris`	GET	Sun and Moon positions
`/time/status`	GET	Time synchronisation status

5.4.2 Control endpoints

Table 14: HTTP control endpoints

Endpoint	Parameters	Description
`/goto`	`ra`, `dec`	Slew to RA/Dec (J2000)
`/goto/galactic`	`l`, `b`	Slew to galactic coordinates
`/track/radec`	`ra`, `dec`	Track RA/Dec (J2000)
`/track/galactic`	`l`, `b`	Track galactic coordinates
`/track/sun`	—	Track Sun
`/track/moon`	—	Track Moon
`/track`	`enable=0/1`	Enable/disable tracking
`/direct`	`alt`, `az`	Go to Alt/Az directly

5.4.3 Example API usage

```python
import requests

ESP32_IP = "192.168.4.1"

def track_source(ra_hours, dec_deg):
    """Start tracking an RA/Dec position (J2000)"""
    r = requests.get(f"http://{ESP32_IP}/track/radec",
                     params={"ra": ra_hours, "dec": dec_deg})
    return r.json()

def track_galactic(l_deg, b_deg):
    """Track galactic coordinates"""
    r = requests.get(f"http://{ESP32_IP}/track/galactic",
                     params={"l": l_deg, "b": b_deg})
    return r.json()

def get_status():
    return requests.get(f"http://{ESP32_IP}/status").json()

# Example: observe Cygnus A
track_source(ra_hours=19.991, dec_deg=40.734)
```

Listing 3: Python API client example

5.5 Stellarium protocol

The ESP32 implements the Stellarium telescope control protocol on TCP port 10001.

5.5.1 Protocol format

Goto Command (from Stellarium): 20 bytes with message type 0, RA/Dec as unsigned 32-bit integers

Position Report (to Stellarium): 24 bytes with current position in same format

5.5.2 Stellarium configuration

1. Open **Configuration** → **Plugins** → **Telescope Control**
2. Enable the plugin and restart Stellarium

3. Add telescope with settings:
    - Type: “External software or remote computer”
    - Host: ESP32 IP address (e.g., 192.168.4.1)
    - Port: 10001
4. Click Connect
5. Select any object and press Ctrl+1 to slew

5.6 Tracking mode

When tracking is enabled, the ESP32:

1. Converts current RA/Dec to Alt/Az using current sidereal time
2. Updates calculation every 1 second
3. For Sun/Moon, refreshes ephemeris every 30 seconds
4. Checks mount limits before sending commands
5. Sends updated position to Arduino Due

5.6.1 Below-horizon handling

When a tracked target drops below the horizon (altitude $< 0°$):

1. Telescope parks at home position
2. System enters “waiting for rise” state
3. Position calculation continues each second
4. When target rises above $0°$, tracking resumes automatically

5.6.2 Azimuth wrap-around

For circumpolar sources that exit the azimuth limits:

1. Tracking enters “waiting” state when azimuth exceeds limits
2. Position calculation continues without sending commands
3. When source reappears within valid range, tracking resumes

6 Coordinate transformations

6.1 Reference frame

All equatorial coordinates use the **J2000.0 reference frame**:

- Standard epoch J2000.0 (January 1, 2000, 12:00 TT)
- Equinox J2000.0
- Compatible with modern star catalogues (Hipparcos, Tycho-2)
- Used by Stellarium, SIMBAD, and most planetarium software
- Equivalent to ICRS to within milliarcseconds

6.2 Coordinate conversion pipeline

The transformation from celestial to horizontal coordinates proceeds:

$$\text{RA/Dec (J2000)} \xrightarrow{\text{precess}} \text{RA/Dec (date)} \xrightarrow{\text{HA}} \text{Alt/Az} \tag{4}$$

6.2.1 Julian date calculation

The Julian Date is computed from the UTC calendar date:

$$\text{JD} = \lfloor 365.25(Y + 4716) \rfloor + \lfloor 30.6001(M + 1) \rfloor + D + B - 1524.5 + \frac{h + m/60 + s/3600}{24} \tag{5}$$

where Y is the year, M is the month (with January and February counted as months 13 and 14 of the previous year), D is the day, $B = 2 - \lfloor Y/100 \rfloor + \lfloor Y/400 \rfloor$, and h, m, s are UTC hours, minutes, seconds.

6.2.2 Greenwich mean sidereal time

GMST in hours (IAU 1982 formula):

$$\text{GMST} = 6.697374558 + 0.06570982441908 D_0 + 1.00273790935 H + 0.000026 T^2 \tag{6}$$

where D_0 is the number of days since J2000.0 at 0h UT, H is the hours since 0h UT, and $T = D_0/36525$.

6.2.3 Local sidereal time

$$\text{LST} = \text{GMST} + \frac{\lambda}{15} \tag{7}$$

where λ is the observer's longitude in degrees (east positive).

6.3 Precession

The controller applies IAU 1976 precession to convert J2000 coordinates to the equinox of the current date. The precession angles in arcseconds:

$$\zeta_A = (2306.2181 + 1.39656T)T + 0.30188T^2 + 0.017998T^3 \tag{8}$$

$$z_A = (2306.2181 + 1.39656T)T + 1.09468T^2 + 0.018203T^3 \tag{9}$$

$$\theta_A = (2004.3109 - 0.85330T)T - 0.42665T^2 - 0.041833T^3 \tag{10}$$

where $T = (\text{JD} - 2451545.0)/36525$ is the Julian centuries from J2000.0.
The precession rotation is then applied:

$$A = \cos\delta_0 \sin(\alpha_0 + \zeta) \tag{11}$$

$$B = \cos\theta \cos\delta_0 \cos(\alpha_0 + \zeta) - \sin\theta \sin\delta_0 \tag{12}$$

$$C = \sin\theta \cos\delta_0 \cos(\alpha_0 + \zeta) + \cos\theta \sin\delta_0 \tag{13}$$

with new coordinates:

$$\alpha = \arctan 2(A, B) + z \tag{14}$$

$$\delta = \arcsin(C) \tag{15}$$

6.4 Horizontal coordinates

The hour angle is computed:

$$\mathrm{HA} = \mathrm{LST} - \alpha \tag{16}$$

where α is the right ascension in hours.
The altitude is:

$$\sin h = \sin\delta \sin\phi + \cos\delta \cos\phi \cos \mathrm{HA} \tag{17}$$

The azimuth is:

$$\cos A = \frac{\sin\delta - \sin h \sin\phi}{\cos h \cos\phi} \tag{18}$$

with quadrant correction based on $\sin \mathrm{HA}$:

$$A = \begin{cases} \arccos(A) & \text{if } \sin \mathrm{HA} \leq 0 \\ 2\pi - \arccos(A) & \text{if } \sin \mathrm{HA} > 0 \end{cases} \tag{19}$$

where ϕ is the observer's latitude.

6.5 Galactic coordinates

Galactic to equatorial conversion uses the J2000 galactic coordinate system constants:

$$\alpha_{\mathrm{NGP}} = 192.85948^\circ \quad \text{(RA of North Galactic Pole)} \tag{20}$$

$$\delta_{\mathrm{NGP}} = 27.12825^\circ \quad \text{(Dec of North Galactic Pole)} \tag{21}$$

$$l_{\mathrm{NCP}} = 122.93192^\circ \quad \text{(Galactic longitude of North Celestial Pole)} \tag{22}$$

The conversion equations:

$$\sin\delta = \sin\delta_{\mathrm{NGP}} \sin b + \cos\delta_{\mathrm{NGP}} \cos b \cos(l_{\mathrm{NCP}} - l) \tag{23}$$

$$\alpha = \alpha_{\mathrm{NGP}} + \arctan 2(\cos b \sin(l_{\mathrm{NCP}} - l), \cos\delta_{\mathrm{NGP}} \sin b - \sin\delta_{\mathrm{NGP}} \cos b \cos(l_{\mathrm{NCP}} - l)) \tag{24}$$

6.6 Solar position

The Sun's position is calculated using the algorithm from Meeus "Astronomical Algorithms":

1. Mean longitude: $L_0 = 280.46646 + 36000.76983T$
2. Mean anomaly: $M = 357.52911 + 35999.05029T$
3. Equation of centre: $C = 1.914602 \sin M + 0.019993 \sin 2M + 0.000289 \sin 3M$
4. True longitude: $\lambda = L_0 + C$
5. Convert ecliptic to equatorial coordinates
6. Precess from apparent (equinox of date) to J2000

Accuracy: approximately 1 arcminute.

6.7 Lunar position

The Moon's position uses a simplified version of the ELP/MPP02 theory:

1. Calculate mean orbital elements (L', D, M, M', F)
2. Apply periodic terms for longitude and latitude
3. Convert geocentric ecliptic to equatorial
4. Precess to J2000

Accuracy: approximately 5–10 arcminutes (adequate for pointing a radio telescope with 0.5° resolution).

6.8 Pointing accuracy

The coordinate transformations achieve < 0.5 arcminute RMS error compared to ERFA/astropy for typical observing scenarios. Larger errors may occur:

- Near zenith (azimuth becomes ill-defined)
- At very low altitudes (atmospheric refraction not modelled)
- For the Moon (simplified orbital model)

7 H1 receiver and observation scheduler

7.1 Receiver overview

The H1 receiver (`b210_h1_receiver.py`) is a GNU Radio-based spectrum analyser for 21 cm hydrogen line observations:

- Real-time FFT processing with configurable integration time
- Live spectrum and waterfall display (PyQtGraph)
- Continuous data logging to HDF5 format
- Support for Ettus B210, RTL-SDR, or demo mode

7.2 Supported hardware

Table 15: Supported SDR platforms

SDR	Frequency Range	Sample Rate	Notes
Ettus B210	70 MHz – 6 GHz	Up to 56 MHz	Recommended
RTL-SDR (R820T)	24 – 1766 MHz	Up to 2.4 MHz	Budget option
Demo mode	—	—	Simulated data

7.3 Command line options

```
python b210_h1_receiver.py --sdr b210 --gain 40 --sample-rate 2.4e6
python b210_h1_receiver.py --sdr rtlsdr --gain 45
python b210_h1_receiver.py --sdr demo
```

Listing 4: Receiver command line usage

7.4 HDF5 output format

The receiver outputs data in HDF5 format with the following structure:

Table 16: HDF5 dataset structure

Dataset	Shape	Description
`frequency_hz`	$(N_{\text{fft}},)$	Frequency axis in Hz
`spectra_db`	$(N_{\text{spectra}}, N_{\text{fft}})$	Power spectra in dB
`timestamps`	$(N_{\text{spectra}},)$	Unix timestamps
`integration_times`	$(N_{\text{spectra}},)$	Actual integration per spectrum

File attributes include SDR type, centre frequency, sample rate, FFT size, gain, and creation timestamp.

7.5 Observation scheduler

The web scheduler (`h1_web_scheduler.py`) coordinates telescope pointing and data acquisition:

- Web-based interface for creating observation schedules
- Automatic telescope pointing via HTTP API
- Receiver process management
- Persistent schedule storage (JSON)

7.5.1 Starting the scheduler

```
python h1_web_scheduler.py --host 0.0.0.0 --port 5000
```

Then open `http://localhost:5000` in a browser.

7.5.2 Observation parameters

Table 17: Observation schedule parameters

Parameter	Description
Name	Descriptive name
Start Date/Time	When to start (local time)
Duration	Observation length (minutes)
Coordinates	Target position (Alt/Az, RA/Dec, or Galactic)
Centre Frequency	Observation frequency (default: 1420.405 MHz)
Bandwidth	Sample rate / bandwidth (MHz)
Gain	RF gain (dB)
Channels	FFT size
Integration Time	Seconds per averaged spectrum
SDR Type	B210, RTL-SDR, or Demo

7.5.3 Automated workflow

1. User creates observation via web interface

2. At scheduled time, scheduler sends HTTP request to ESP32
3. ESP32 converts coordinates and commands Arduino Due
4. Telescope slews to target
5. Scheduler launches GNU Radio receiver
6. Data saved to HDF5 with timestamps and metadata

8 Installation

8.1 Prerequisites

- PlatformIO (VS Code extension or CLI)
- Python 3.x with `esptool` and `mpremote`
- Radioconda (for receiver functionality)

```
pip install platformio esptool mpremote
```

8.2 Arduino Due Firmware

8.2.1 Building and uploading

```
cd new_SRT_drive

# Build for real hardware
pio run -e due

# Upload to Arduino Due
pio run -e due -t upload

# Monitor serial output
pio device monitor -b 115200
```

8.2.2 Simulation mode

```
# Build for simulation
pio run -e simulation

# Upload simulation firmware
pio run -e simulation -t upload
```

8.3 ESP32-S3 controller

8.3.1 Flashing MicroPython

Download firmware from https://micropython.org/download/ESP32_GENERIC_S3/

```
# Erase flash (hold BOOT button while connecting if needed)
esptool.py --chip esp32s3 --port COM5 erase_flash

# Flash MicroPython
esptool.py --chip esp32s3 --port COM5 write_flash -z 0 ESP32_GENERIC_S3-*.bin
```

8.3.2 Uploading controller code

```
cd new_SRT_drive/esp32_controller

# Upload all Python files
mpremote connect COM5 cp *.py :

# Upload help page
mpremote connect COM5 cp help.html :

# Reset to run
mpremote connect COM5 reset
```

8.4 Configuration

8.4.1 Observer location

Edit `esp32_controller/config.py` before uploading:

```
# Critical: set to your observatory location
OBSERVER_LAT = 55.9    # Latitude (degrees, north positive)
OBSERVER_LON = -4.3    # Longitude (degrees, west negative)
```

8.4.2 Receiver prerequisites

```
# Install Radioconda from:
# https://github.com/ryanvolz/radioconda/releases

# Or install dependencies manually:
conda install gnuradio gnuradio-uhd pyqtgraph h5py flask
```

9 Operation

9.1 Startup sequence

1. Power on both controllers (Arduino Due and ESP32)
2. Arduino Due performs automatic homing sequence:
   (a) Drives to limit switches
   (b) Resets position counters
   (c) Moves to home position (Alt=0°, Az=180°)
3. ESP32 starts WiFi access point
4. ESP32 attempts NTP time synchronisation
5. System ready when Due reports “Ready” status

The homing sequence takes approximately 30–60 seconds depending on starting position.

9.2 Web interface access

9.2.1 Direct WiFi connection

1. Connect to WiFi network: `SRT_Controller`
2. Password: `radio1420`
3. Browse to: `http://192.168.4.1`

9.2.2 Network connection

1. Connect to AP first
2. Go to WiFi tab, scan for networks
3. Select network and enter password
4. Note the assigned IP address
5. Connect your computer to the same network
6. Browse to the new IP address

9.3 Tracking observations

1. Enter coordinates in the appropriate format:
   - RA/Dec: Right Ascension (0–24 hours), Declination (−90 to +90 degrees)
   - Galactic: Longitude l (0–360°), Latitude b (−90 to +90°)
   - Alt/Az: Altitude (0–90°), Azimuth (0–355°)
2. Click “Track” for continuous tracking or “Go To” for single slew
3. Monitor status display for position and tracking state

9.4 Quick targets

The web interface provides one-click tracking for:

- **Sun** — Automatic ephemeris calculation
- **Moon** — Automatic ephemeris calculation
- **Home** — Return to home position

9.5 Time synchronisation

Accurate time is required for coordinate calculations:

NTP (primary): Automatic synchronisation from internet time servers

Browser fallback: If NTP fails, the web interface sends browser time automatically

Time status is displayed on the Control tab.

9.6 Safety features

- **Position limits:** Software and hardware limits prevent over-travel
- **Current limiting:** Motors stop on overcurrent
- **Stall detection:** Motors stop if position doesn't change
- **Fault state:** Requires explicit reset to recover

10 Troubleshooting

10.1 System does not start

Symptom	Check
No serial output	USB cable, correct port selected
Stuck in homing	Motor connections, encoder connections
Immediate fault	Motor driver power, fault flag wiring

10.2 Motors do not move

Symptom	Check
PWM output but no motion	Motor driver reset pins (should be HIGH)
No PWM output	PWM pin connections (pins 8, 10)
Motors run backwards	Direction pin wiring (pins 9, 11)

10.3 Position errors

Symptom	Check
Wrong position after homing	Home offset configuration
Position drifts	Encoder debounce (increase DEBOUNCE_MS)
Overshoots target	RAMPDOWN value (increase)
Jerky motion	RAMPUP value (increase)

10.4 Web interface issues

Symptom	Check
Can't connect to 192.168.4.1	WiFi connection to SRT_Controller
Page loads but no data	Due serial connection, baud rate
Buttons don't respond	Browser console for JavaScript errors

10.5 Coordinate issues

Symptom	Check
Position doesn't match sky	OBSERVER_LAT/LON in config.py
Large errors (>1°)	Time sync status
Sun/Moon wrong by ∼20 arcmin	Normal for simplified ephemeris

10.6 Stellarium connection issues

Symptom	Check
Can't connect	IP address and port 10001
Connects but no slew	Click object then Ctrl+1
Wrong position shown	Time synchronisation

11 Conclusion

The SRT Drive Controller provides a complete, modular solution for controlling a small radio telescope for 21 cm hydrogen line observations. The layered architecture separates real-time motor control from network interface and astronomical computations, enabling reliable operation and straightforward maintenance.

Key features include:

- Sub-degree positioning accuracy (0.5° resolution)
- J2000 coordinate system compatibility with modern catalogues and planetarium software
- Multiple control interfaces (web, Stellarium, HTTP API)
- Automated scheduling and data acquisition
- Comprehensive safety features
- Simulation mode for testing without hardware

The system is designed for educational and amateur radio astronomy applications, providing a practical platform for hydrogen line observations and radio astronomy instruction.

A Pin quick reference

```
Motor Control:  8(PWM-Az), 9(DIR-Az), 10(PWM-Alt), 11(DIR-Alt)
Driver Reset:   22(Az), 24(Alt)
Encoders:       12(Az), 13(Alt)
Fault Flags:    4,5(Az), 6,7(Alt)
Current Sense:  A0(Alt), A1(Az)
Serial1:        18(TX), 19(RX)
```

Listing 5: Arduino Due pin summary

B Command quick reference

```
45 180            # Slew to Alt=45, Az=180
HOME              # Run homing sequence
STOP              # Emergency stop
RESET             # Clear fault and re-home
STATUS            # Show current status
CONFIG            # Show configuration
SET AZMAX 350     # Set parameter
SAVE              # Save to flash
HELP              # Show all commands
```

Listing 6: Common serial commands

C HTTP API quick reference

```
# Track RA/Dec (J2000)
curl "http://192.168.4.1/track/radec?ra=12&dec=30"

# Track galactic coordinates
curl "http://192.168.4.1/track/galactic?l=0&b=0"

# Track Sun
curl "http://192.168.4.1/track/sun"

# Stop tracking
curl "http://192.168.4.1/track?enable=0"

# Get status
curl "http://192.168.4.1/status"
```

Listing 7: HTTP API examples